

Name: Shawon Gazi

ID: 241400031

Course Name: Assembly Language Lab

Problem 1: Positive or Negative Checker

Write a program that takes a single-digit number as input and displays whether it is Positive or Negative using conditional jumps.

Solution(.asm):

```
1. .MODEL SMALL
2. .STACK 100H
3.
4. .DATA
5.     prompt DB "Enter a number (-9 to 9): $"
6.     posMsg DB 13,10,"The number is Positive.$"
7.     negMsg DB 13,10,"The number is Negative.$"
8.     zeroMsg DB 13,10,"The number is Zero.$"
9.     num DB ?
10.
11.     .CODE
12.     MAIN PROC
13.         MOV AX,@DATA
14.         MOV DS,AX
15.
16.         ; Display prompt
17.         MOV AH,09H
18.         LEA DX,prompt
19.         INT 21H
20.
21.         ; Read signed number (single-digit)
22.         MOV AH,01H
23.         INT 21H
24.         ; Check for negative sign
25.         CMP AL,'-'
26.         JNE POSITIVE_CHECK
27.         ; Negative detected
28.         MOV AH,01H
29.         INT 21H
30.         SUB AL,'0'
31.         NEG AL
32.         JMP CHECK
33.
34.     POSITIVE_CHECK:
35.         SUB AL,'0'
36.
37.     CHECK:
38.         MOV num,AL
39.         CMP AL,0
```

```

40.         JG POSITIVE
41.         JL NEGATIVE
42.         JE ZERO
43.
44. POSITIVE:
45.         MOV AH,09H
46.         LEA DX,posMsg
47.         INT 21H
48.         JMP EXIT
49.
50. NEGATIVE:
51.         MOV AH,09H
52.         LEA DX,negMsg
53.         INT 21H
54.         JMP EXIT
55.
56. ZERO:
57.         MOV AH,09H
58.         LEA DX,zeroMsg
59.         INT 21H
60.
61. EXIT:
62.         MOV AH,4CH
63.         INT 21H
64.
65. MAIN ENDP
66. END MAIN

```

```

ASM pos_neg_checker.asm > ...
1  1  .MODEL SMALL
2  2  .STACK 100H
3
4  3  .DATA
5      prompt DB "Enter a number (-9 to 9): $"
6      posMsg DB 13,10,"The number is Positive.$"
7      negMsg DB 13,10,"The number is Negative.$"
8      zeroMsg DB 13,10,"The number is Zero.$"
9      num DB ?
10
11 4  .CODE
12 5  MAIN PROC
13 6      MOV AX,@DATA
14 7      MOV DS,AX
15
16 8      ; Display prompt
17 9      MOV AH,09H
18
19

```

```

Z:\>mount d /z/code
Drive D is mounted as local directory

Z:\>d:

D:\>set PATH=C:\IASM

D:\>TASM D:\test.asm
Turbo Assembler Version 4.1 Copyright

Assembling file: D:\test.asm to te
Error messages:  None
Warning messages:  None
Passes: 1
Remaining memory: 467k

D:\>TLINK D:\test
Turbo Link Version 7.1.30.1. Copyright

D:\>D:\test
Enter a number (-9 to 9): 7
The number is Positive.
D:\>IF

```

Problem 2: Problem 2: Vowel or Consonant Checker

Write a program that takes a character as input and determines if it is a vowel (A,E,I,O,U) or consonant using CMP and JE instructions.

Solution(.asm):

```
1. .MODEL SMALL
2. .STACK 100h
3. .DATA
4.     msgVowel DB "Vowel$"
5.     msgCons  DB "Consonant$"
6.
7. .CODE
8. MAIN PROC
9.     MOV AX, @DATA
10.    MOV DS, AX
11.
12.    ; Read a character
13.    MOV AH, 01h
14.    INT 21h
15.    MOV BL, AL      ; Store input character
16.
17.    ; Compare with vowels
18.    CMP BL, 'A'
19.    JE VOWEL
20.    CMP BL, 'E'
21.    JE VOWEL
22.    CMP BL, 'I'
23.    JE VOWEL
24.    CMP BL, 'O'
25.    JE VOWEL
26.    CMP BL, 'U'
27.    JE VOWEL
28.
29.    ; If no match, it's a consonant
30.    MOV DX, OFFSET msgCons
31.    JMP DISPLAY
32.
33. VOWEL:
34.     MOV DX, OFFSET msgVowel
35.
36. DISPLAY:
37.     MOV AH, 09h
38.     INT 21h
39.
40.     ; Exit program
41.     MOV AH, 4ch
42.     INT 21h
43. MAIN ENDP
44. END MAIN
```

The screenshot shows the Turbo Assembler (TASM) interface. The main window displays the assembly code for 'vowel.asm'. The code defines two data strings, 'msgVowel' and 'msgCons', and a procedure 'MAIN' that reads a character and checks if it is a vowel. The output window on the right shows the execution results, including the assembly path, file name, and the output of the program: 'It is Vowel\$'.

```

1 .MODEL SMALL
2 .STACK 100h
3 .DATA
4     msgVowel DB "It is Vowel$"
5     msgCons  DB "It is Consonant$"
6
7 .CODE
8 MAIN PROC
9     MOV AX, @DATA
10    MOV DS, AX
11
12    ; Read a character
13    MOV AH, 01h
14    INT 21h
15    MOV BL, AL    ; Store input character
16
17    ; Compare with vowels
18    CMP BL, 'A'
19    JE VOWEL
20    CMP BL, 'E'
21    JE VOWEL
22    CMP BL, 'I'
23    JE VOWEL
24    CMP BL, 'O'
25    JE VOWEL
26    CMP BL, 'U'
27    JE VOWEL
28
29    ; If no match, it's a consonant
30    MOV DX, OFFSET msgCons
31    JMP DISPLAY
32
33 END MAIN
34
35
```

Output window:

```

D:\>TASM PATH-C:\IASM
D:\>TASM D:\test.asm
Turbo Assembler Version 4.1 Copyright (c) 1988, 1996 Borland International
Assembling file: D:\test.asm to test.OBJ
Warning: D:\test.asm(36) Reserved word used as symbol: DISPLAY
Error messages: None
Warning messages: 1
Passes: 1
Remaining memory: 467k

D:\>TLINK D:\test
Turbo Link Version 7.1.30.1. Copyright (c) 1987, 1996 Borland International
D:\>D:\test
It is Vowel
D:\>

```

Problem 3: Largest of Three Numbers

Write a program that stores three numbers and finds the largest using nested comparisons with JG/JGE instructions.

Solution(.asm):

```

1. .MODEL SMALL
2. .STACK 100H
3.
4. .DATA
5.     num1 DW 7
6.     num2 DW -3
7.     num3 DW 5
8.     largest DW ?
9.
10.     msg DB "The largest number is: $"
11.
12. .CODE
13. MAIN PROC
14.     MOV AX, @DATA
15.     MOV DS, AX
16.
17.     ; Compare num1 and num2
18.     MOV AX, num1

```

```

19.      CMP AX, num2
20.      JGE FIRST_LARGER
21.      MOV AX, num2
22.
23.  FIRST_LARGER:
24.      ; AX now holds larger of num1 and num2
25.
26.      ; Compare with num3
27.      CMP AX, num3
28.      JGE STORE_LARGEST
29.      MOV AX, num3
30.
31.  STORE_LARGEST:
32.      MOV largest, AX
33.
34.      ; Display message
35.      MOV AH, 09H
36.      LEA DX, msg
37.      INT 21H
38.
39.      ; Display number (simple single digit assumption)
40.      MOV AX, largest
41.      CALL PRINT_NUM
42.
43.      ; Exit
44.      MOV AH, 4CH
45.      INT 21H
46.
47.  MAIN ENDP
48.
49.  ; ----- PRINT NUMBER PROCEDURE -----
50.  PRINT_NUM PROC
51.      CMP AX, 0
52.      JGE POSITIVE
53.      ; print minus sign
54.      MOV DL, '-'
55.      MOV AH, 02H
56.      INT 21H
57.      NEG AX
58.
59.  POSITIVE:
60.      ADD AX, 30H
61.      MOV DL, AL
62.      MOV AH, 02H
63.      INT 21H
64.      RET
65.  PRINT_NUM ENDP
66.
67.  END MAIN
68.

```

```

ASM largest_numbr.asm > CODE > PRINT_NUM
1 .MODEL SMALL
2 .STACK 100H
3
4 .DATA
5     num1 DW 7
6     num2 DW -3
7     num3 DW 5
8     largest DW ?
9
10    msg DB "The largest number is: $"
11
12 .CODE
13 MAIN PROC
14     MOV AX, @DATA
15     MOV DS, AX
16
17     ; Compare num1 and num2
18     MOV AX, num1
19     CMP AX, num2
20     JGE FIRST_LARGER
21     MOV AX, num2
22

```

```

jsdos10:15:38 AM X
[ ] pause [x] sound Worker
Drive C is mounted as local directory ./
Z:\>mount d ./code
Drive D is mounted as local directory ./code/
Z:\>d:
D:\>set PATH=C:\TASM
D:\>TASM D:\test.asm
Turbo Assembler Version 4.1 Copyright (c) 1988, 1996 B
Assembling file: D:\test.asm to test.OBJ
Error messages: None
Warning messages: None
Passes: 1
Remaining memory: 467k
D:\>TLINK D:\test
Turbo Link Version 7.1.30.1. Copyright (c) 1987, 1996 B
D:\>D:\test
The largest number is: 7
D:\>

```

Problem 4: Simple Calculator

Write a menu-driven program that takes operation choice (1-2) and two numbers, performs the operation (Add/Sub), and displays result.

Solution(.asm)

```

1. .MODEL SMALL
2. .STACK 100H
3. .DATA
4.     menu DB 13,10,"select operation:",13,10,"1. Add",13,10,"2.
    Subtract",13,10,"$"
5.     prompt1 DB 13,10,"Enter first number (0-9): $"
6.     prompt2 DB 13,10,"Enter second number (0-9): $"
7.     resMsg DB 13,10,"Result: $"
8.     choice DB ?
9.     num1 DB ?
10.    num2 DB ?
11.    result DB ?
12.
13. .CODE
14. MAIN PROC
15.     MOV AX,@DATA
16.     MOV DS,AX
17.
18.     ; Display menu
19.     MOV AH,09H

```

```

20.      LEA DX,menu
21.      INT 21H
22.
23.      ; Read choice
24.      MOV AH,01H
25.      INT 21H
26.      SUB AL,30H          ; Convert ASCII to number
27.      MOV choice,AL
28.
29.      ; Input first number
30.      MOV AH,09H
31.      LEA DX,prompt1
32.      INT 21H
33.      MOV AH,01H
34.      INT 21H
35.      SUB AL,30H
36.      MOV num1,AL
37.
38.      ; Input second number
39.      MOV AH,09H
40.      LEA DX,prompt2
41.      INT 21H
42.      MOV AH,01H
43.      INT 21H
44.      SUB AL,30H
45.      MOV num2,AL
46.
47.      ; Perform operation
48.      MOV AL,choice
49.      CMP AL,1
50.      JE DO_ADD
51.      CMP AL,2
52.      JE DO_SUB
53.      JMP EXIT          ; Invalid choice
54.
55.      DO_ADD:
56.      MOV AL,num1
57.      ADD AL,num2
58.      MOV result,AL
59.      JMP DISPLAY
60.
61.      DO_SUB:
62.      MOV AL,num1
63.      SUB AL,num2
64.      MOV result,AL
65.
66.      DISPLAY:
67.      ; Display message
68.      MOV AH,09H
69.      LEA DX,resMsg
70.      INT 21H
71.
72.      ; Display result (assume <10 for simplicity)
73.      MOV AL,result
74.      ADD AL,30H          ; Convert number to ASCII
75.      MOV DL,AL
76.      MOV AH,02H
77.      INT 21H

```

```

78.
79.     EXIT:
80.         MOV AH,4CH
81.         INT 21H
82.
83.     MAIN ENDP
84.     END MAIN

```

The screenshot shows two windows from the Turbo Assembler environment. The left window, titled 'calc.asm', displays the assembly code. The right window shows the output of the assembly and linking processes.

```

ASM calc.asm > DATA
1  .MODEL SMALL
2  .STACK 100H
3  .DATA
4      menu DB 13,10,"Select operation:",13,10,"1.
5      prompt1 DB 13,10,"Enter first number (0-9):
6      prompt2 DB 13,10,"Enter second number (0-9):
7      resMsg DB 13,10,"Result: $"
8      choice DB ?
9      num1 DB ?
10     num2 DB ?
11     result DB ?
12
13 .CODE
14 MAIN PROC
15     MOV AX,@DATA
16     MOV DS,AX
17
18     ; Display menu
19     MOV AH,09H

```

The right window shows the following output:

```

D:\>TASM D:\test.asm
Turbo Assembler Version 4.1 Copyright (c) 1989 Intel Corp.

Assembling file: D:\test.asm to
Warning: D:\test.asm(11) Reserved
Error messages: None
Warning messages: 1
Passes: 1
Remaining memory: 467k

D:\>TLINK D:\test
Turbo Link Version 7.1.00.1 Copyright (c) 1989 Intel Corp.

D:\>D:\test

Select operation:
1. Add
2. Subtract
1
Enter first number (0-9): 2
Enter second number (0-9): 3
Result: 5
D:\>

```

Problem 5: Grade Calculator

Write a program that takes two-digit marks (0-100), validates input, and displays grade: A(80+), B(70-79), C(60-69), D(50-59), F(0-49).

Solution(.asm)

```

1. .MODEL SMALL
2. .STACK 100H
3.
4. .DATA
5.     prompt DB "Enter marks (0-100): $"
6.     invalid DB 13,10,"Invalid input!$"
7.     gradeA DB 13,10,"Grade: A$"
8.     gradeB DB 13,10,"Grade: B$"
9.     gradeC DB 13,10,"Grade: C$"
10.    gradeD DB 13,10,"Grade: D$"
11.    gradeF DB 13,10,"Grade: F$"
12.    marks DW ?

```



```

13.
14. .CODE
15. MAIN PROC
16.     MOV AX,@DATA
17.     MOV DS,AX
18.
19.     ; Prompt user
20.     MOV AH,09H
21.     LEA DX,prompt
22.     INT 21H
23.
24.     ; Read marks
25.     CALL READ_NUM
26.     MOV marks,AX
27.
28.     ; Validate marks <=100
29.     CMP AX,100
30.     JA INVALID_INPUT
31.
32.     ; Check Grade A (80+)
33.     CMP AX,80
34.     JGE GRADE_A
35.
36.     ; Check Grade B (70-79)
37.     CMP AX,70
38.     JGE GRADE_B
39.
40.     ; Check Grade C (60-69)
41.     CMP AX,60
42.     JGE GRADE_C
43.
44.     ; Check Grade D (50-59)
45.     CMP AX,50
46.     JGE GRADE_D
47.
48.     ; Else Grade F (0-49)
49.     GRADE_F:
50.     MOV AH,09H
51.     LEA DX,gradeF
52.     INT 21H
53.     JMP EXIT
54.
55.     GRADE_D:
56.     MOV AH,09H
57.     LEA DX,gradeD
58.     INT 21H
59.     JMP EXIT
60.
61.     GRADE_C:
62.     MOV AH,09H
63.     LEA DX,gradeC
64.     INT 21H
65.     JMP EXIT
66.
67.     GRADE_B:
68.     MOV AH,09H
69.     LEA DX,gradeB
70.     INT 21H

```

```

71.         JMP EXIT
72.
73.     GRADE_A:
74.         MOV AH,09H
75.         LEA DX,gradeA
76.         INT 21H
77.         JMP EXIT
78.
79.     INVALID_INPUT:
80.         MOV AH,09H
81.         LEA DX,invalid
82.         INT 21H
83.
84.     EXIT:
85.         MOV AH,4CH
86.         INT 21H
87.
88.     MAIN ENDP
89.
90. ; ----- PROCEDURE TO READ NUMBER (0-100) -----
91. READ_NUM PROC
92.     MOV CX,0           ; number
93.     MOV BX,10
94.
95.     READ_LOOP:
96.         MOV AH,01H
97.         INT 21H
98.         CMP AL,13      ; Enter key
99.         JE DONE
100.        SUB AL,30H
101.        MOV AH,0
102.        MUL BX
103.        ADD AX,CX
104.        MOV CX,AX
105.        JMP READ_LOOP
106.
107.     DONE:
108.         MOV AX,CX
109.         RET
110.     READ_NUM ENDP
111.     END MAIN

```

```

ASM grade_calculator.asm > DATA > marks
1 .MODEL SMALL
2 .STACK 100H
3
4 .DATA
5     prompt DB "Enter marks (0-100): $"
6     invalid DB 13,10,"Invalid input!$"
7     gradeA DB 13,10,"Grade: A$"
8     gradeB DB 13,10,"Grade: B$"
9     gradeC DB 13,10,"Grade: C$"
10    gradeD DB 13,10,"Grade: D$"
11    gradeF DB 13,10,"Grade: F$"
12    marks DW ?
13
14 .CODE
15 MAIN PROC
16     MOV AX,@DATA
17     MOV DS,AX
18
19

```

```

jsdos10:46:37 AM X
[ ] pause [x] sound [Worker v]
Z:\>mount d :/code
Drive D is mounted as local directory :/code
Z:\>cd:
D:\>set PATH=C:\TASM
D:\>TASM D:\test.asm
Turbo Assembler Version 1.1 Copyright (c)
Assembling file: D:\test.asm to test.OBJ
Error messages: None
Warning messages: None
Passes: 1
Remaining memory: 466k
D:\>LINK D:\test
Turbo Link Version 7.1.30.1 Copyright (c)
D:\>D:\test
Enter marks (0-100): BZ
Grade: A
D:\>

```

